LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 6. Loops - Control Flow

Lecture: 51. Introduction to Loops

Section 1: Lecture Summary

This lecture introduces the fundamental concept of loops in Python programming, focusing primarily on the while loop. Loops are presented as a core programming feature used to execute repeating statements until a certain condition changes. The instructor explains two main types of loops in Python: the while loop (also called a sentinel loop) and the for loop (a "for each" loop in Python). They highlight that while loops are less commonly used than for loops but are essential for grasping the basics of repetition control. The lecture covers the syntax of the while loop, its control flow, and how to visualize its operation with a flowchart. A simple example program demonstrates using a while loop with a decrementing counter that prints numbers from a starting point down to one. The instructor also emphasizes the importance of understanding loop control mechanisms through tracing program execution and lays the groundwork for further practice and program writing in upcoming lectures.

Section 2: Key Concepts and Explanations

- **Loops in Python**
  - Loops allow repeating execution of statements.
  - Two main types in Python:
    - While loop: Repeats as long as a condition is true (sentinel loop).
    - For loop: Iterates over items in a collection (for each loop in Python).
  - While loop is based on a condition checked before each iteration.

- **While Loop Syntax**

```
while <condition>:
    statement 1
    statement 2
    ...
```

- The indentation denotes the loop body.

- The loop continues executing the indented statements as long as the condition remains true.

- **While Loop Control Flow**

1. Evaluate the condition.

2. If true, execute the loop body statements sequentially.

3. After executing, return to step 1.

4. If false, exit the loop and proceed to the next statement after the loop.

- **Flowchart Representation**

- Start -> Check condition

- If True -> Execute statements -> Loop back to condition

- If False -> Exit loop -> Proceed further

- **Counters in Loops**

- A counter tracks how many times a loop runs.

- Can increment or decrement a value each iteration.

- Example shows a decrementing counter from 5 down to 1.

- **Tracing Loop Execution**

- Tracing involves manually following each step to understand the loop's behavior and output.

- Important for understanding how variables change and when the loop ends.

Section 3: Example Code and Use Cases

```
n = 5
while n > 0:
    print(n)
    n = n - 1
```

- This program initializes a variable `n` with 5.

- It enters a while loop that continues as long as `n` is greater than zero.

- Inside the loop, it prints the current value of `n` and then decrements `n` by 1.

- The loop prints numbers in descending order: 5, 4, 3, 2, 1.

- When `n` becomes zero, the condition `n > 0` is false, and the loop terminates.

Section 4: Key Takeaways

- Loops are essential programming constructs used to repeat statements.

- Python uses mainly two types of loops: while loops (condition-controlled) and for loops (iterable-controlled).

- The while loop executes as long as its condition is true and stops when the condition becomes false.

- Understanding loop flow is crucial; tracing loops helps predict program behavior.

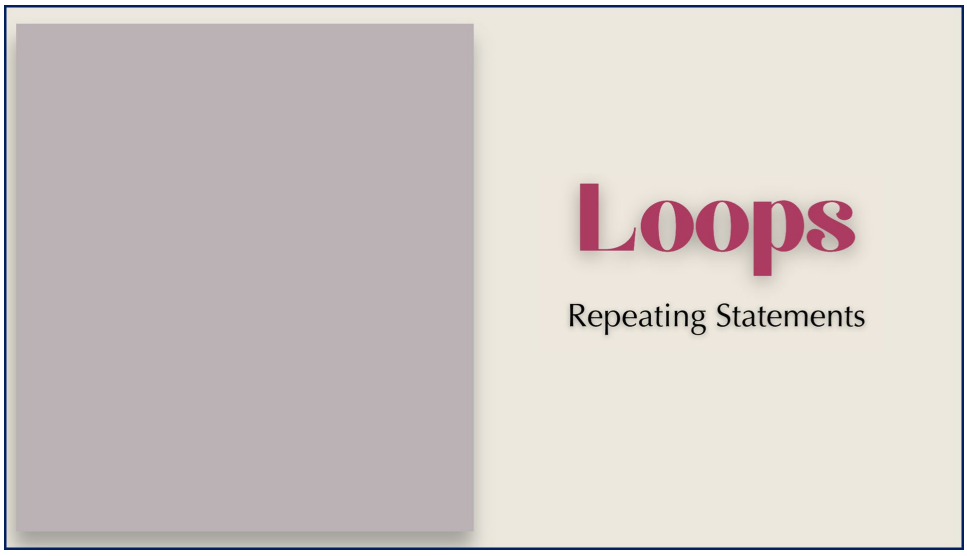
- Counters can be used inside loops to count iterations or control repetition.

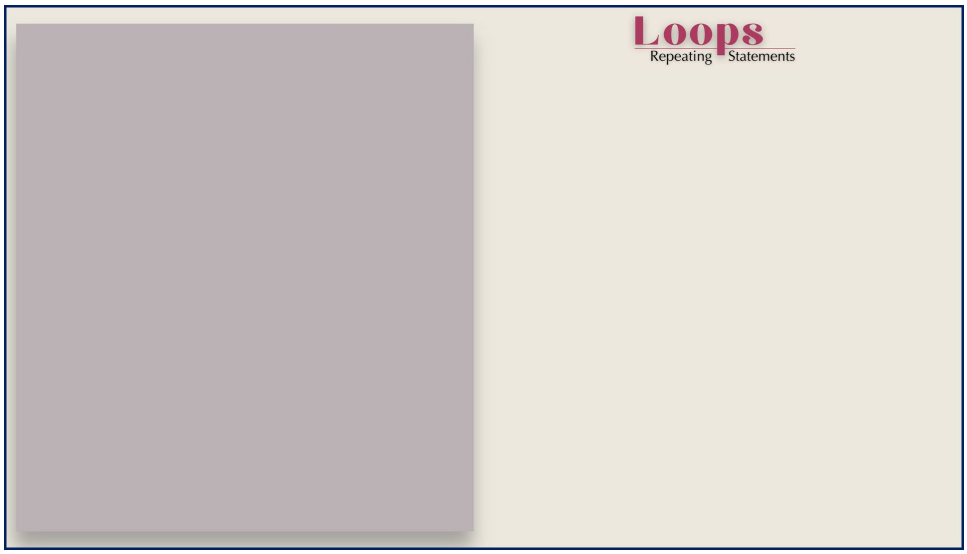
- The provided while loop example demonstrates decrementing a counter and printing values from a starting number down to 1.

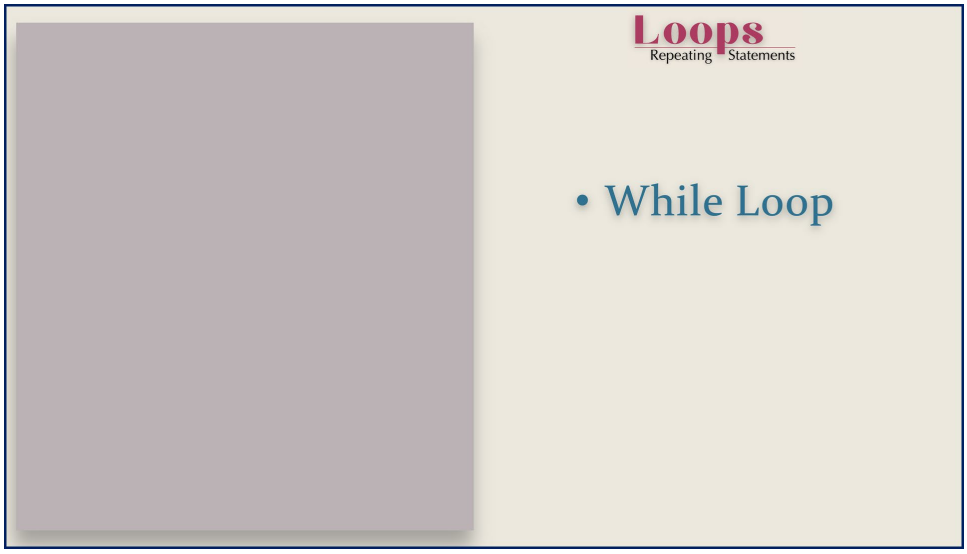
- Mastery of loops is foundational for progressing in programming and solving repetitive tasks effectively.

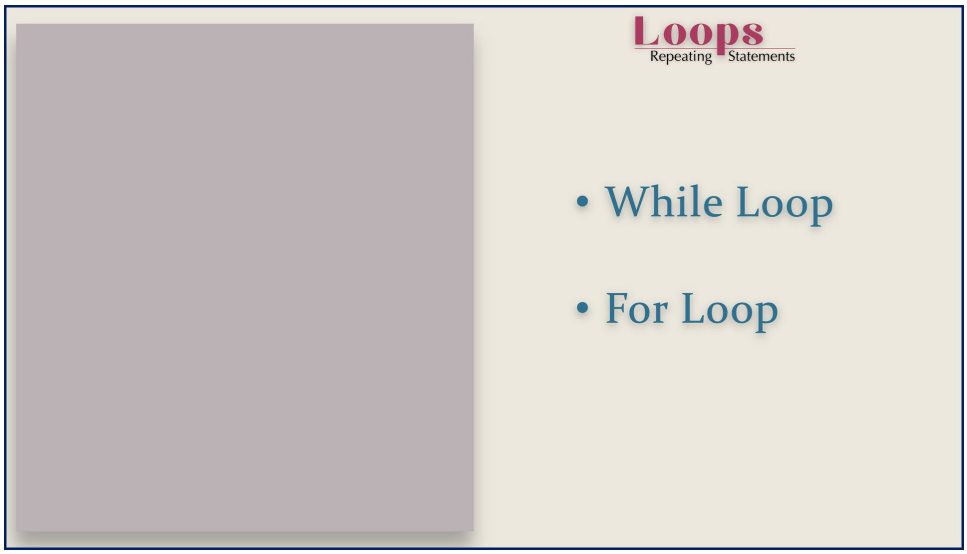
Lecture in Slides

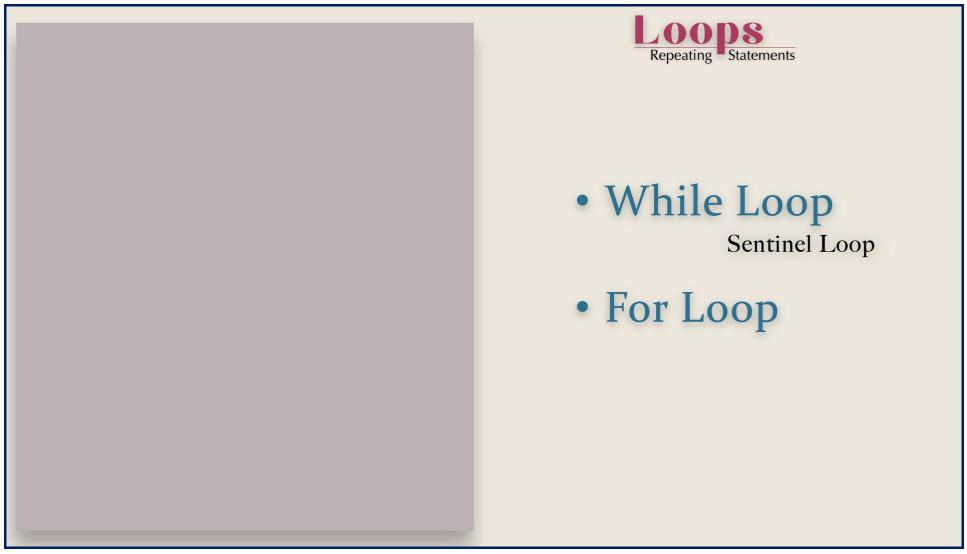
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

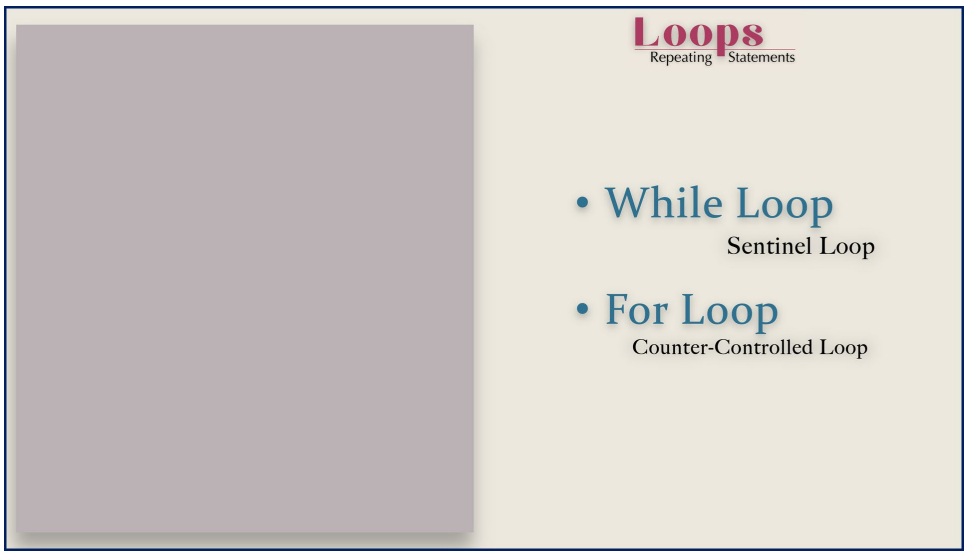


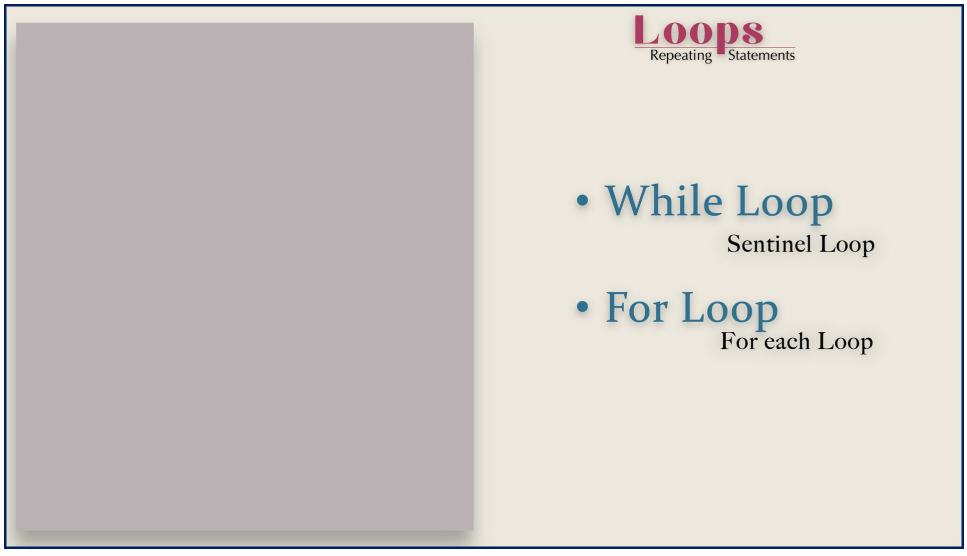

Loops
Repeating Statements


Loops
Repeating Statements
• While Loop

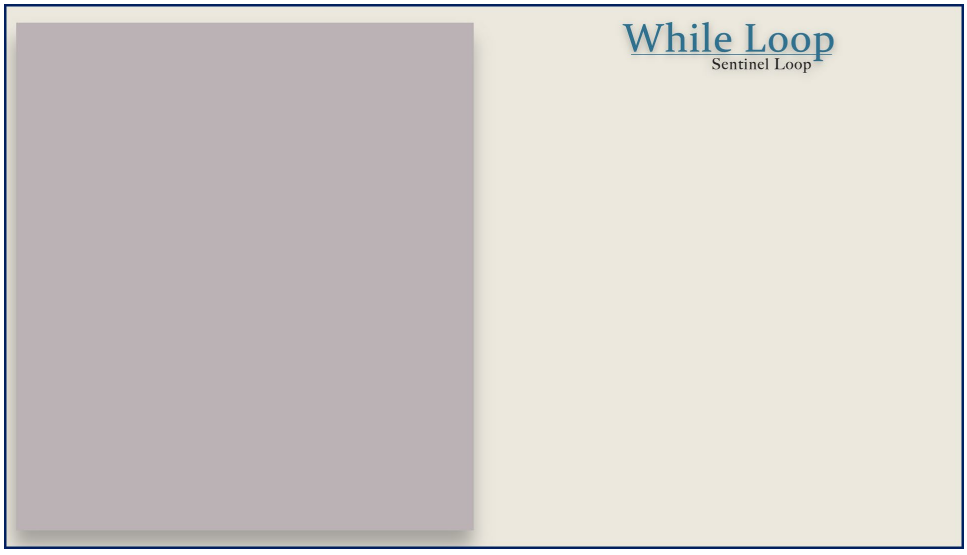

Loops
Repeating Statements
• While Loop
• For Loop

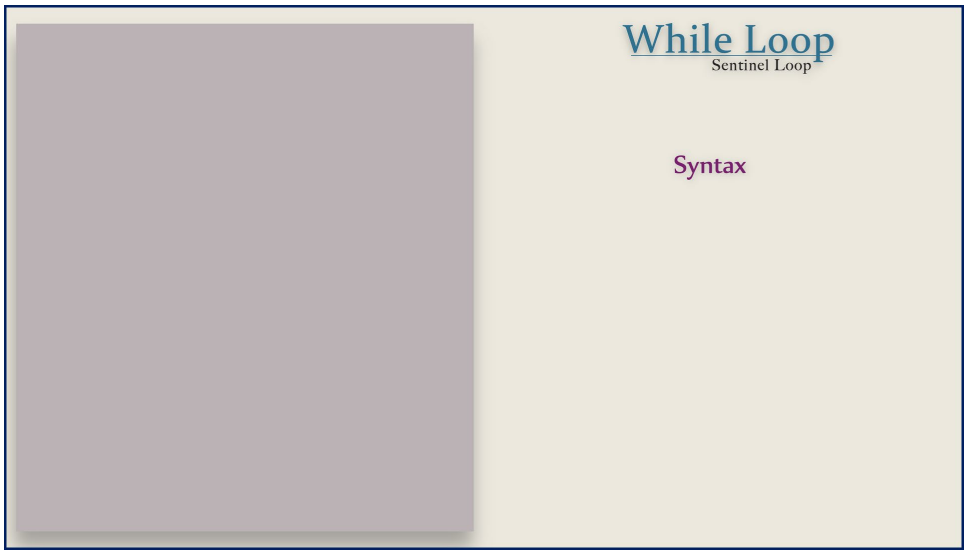

Loops
Repeating Statements
• While Loop
Sentinel Loop
• For Loop

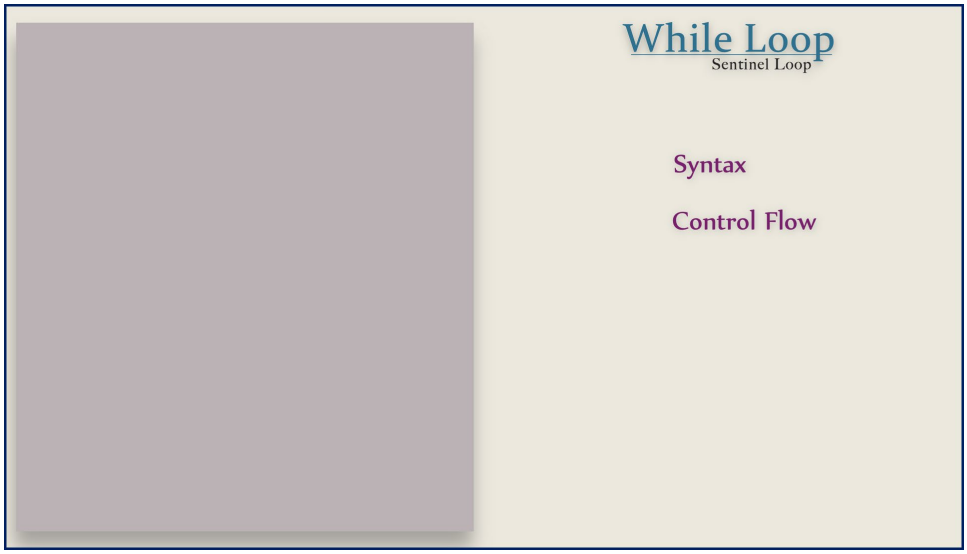

Loops
Repeating Statements
• While Loop
Sentinel Loop
• For Loop
Counter-Controlled Loop


Loops
Repeating Statements
• While Loop
Sentinel Loop
• For Loop
For each Loop


• While Loop
Sentinel Loop


While Loop
Sentinel Loop


While Loop
Sentinel Loop
Syntax


While Loop
Sentinel Loop
Syntax
Control Flow

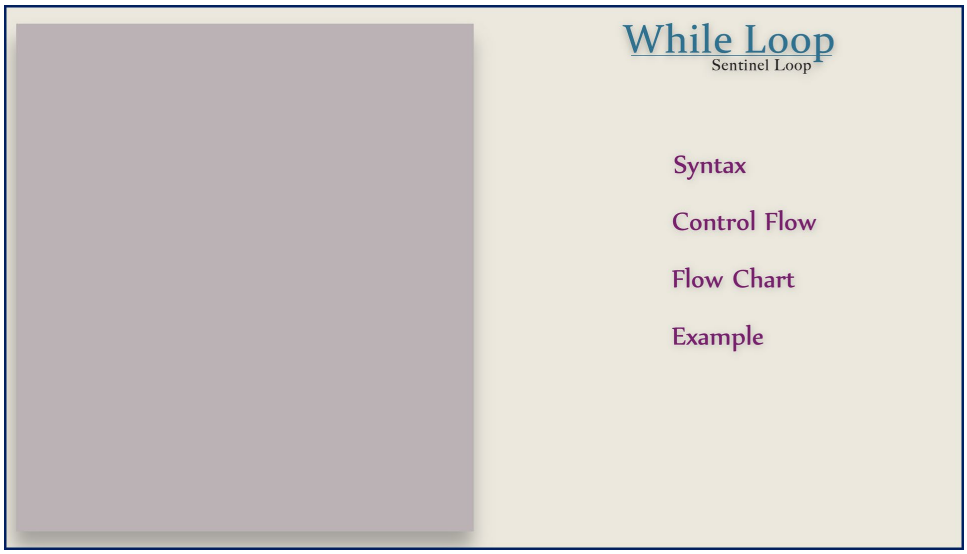
While Loop

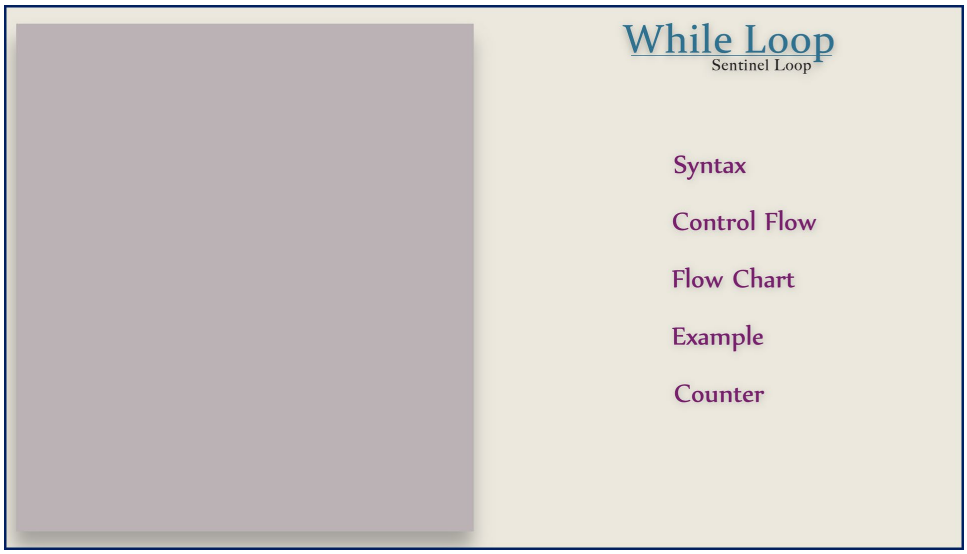
Sentinel Loop

Syntax

Control Flow

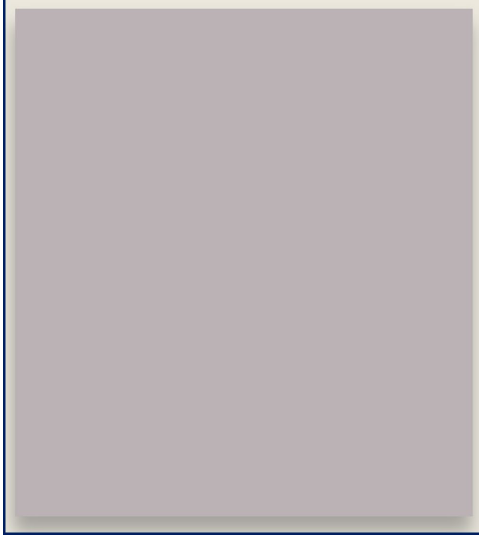
Flow Chart


While Loop
Sentinel Loop
Syntax
Control Flow
Flow Chart
Example


While Loop
Sentinel Loop
Syntax
Control Flow
Flow Chart
Example
Counter

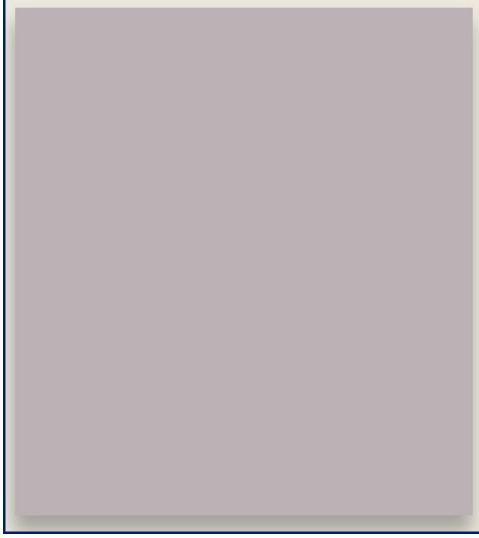
While Loop

Syntax



While Loop

Syntax



While Loop
Syntax

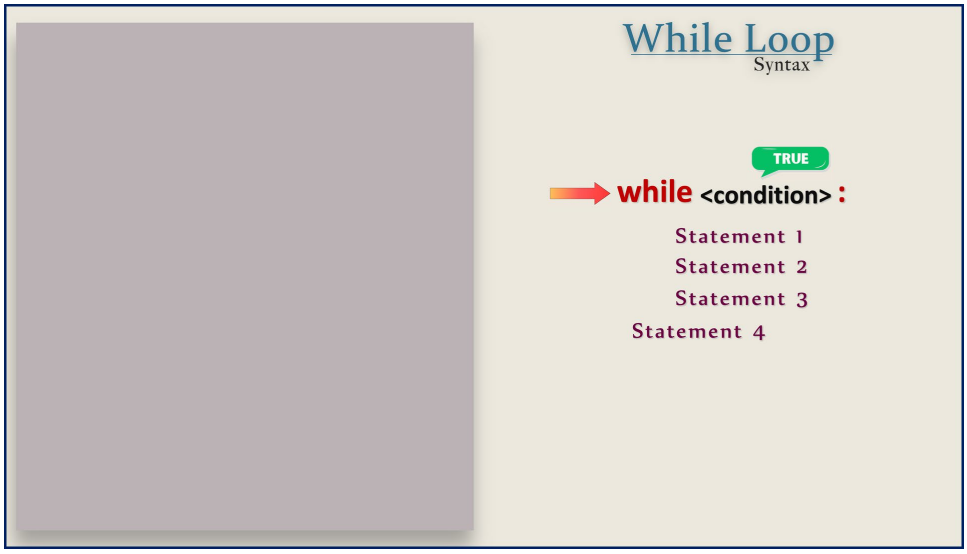
while <condition> **:**

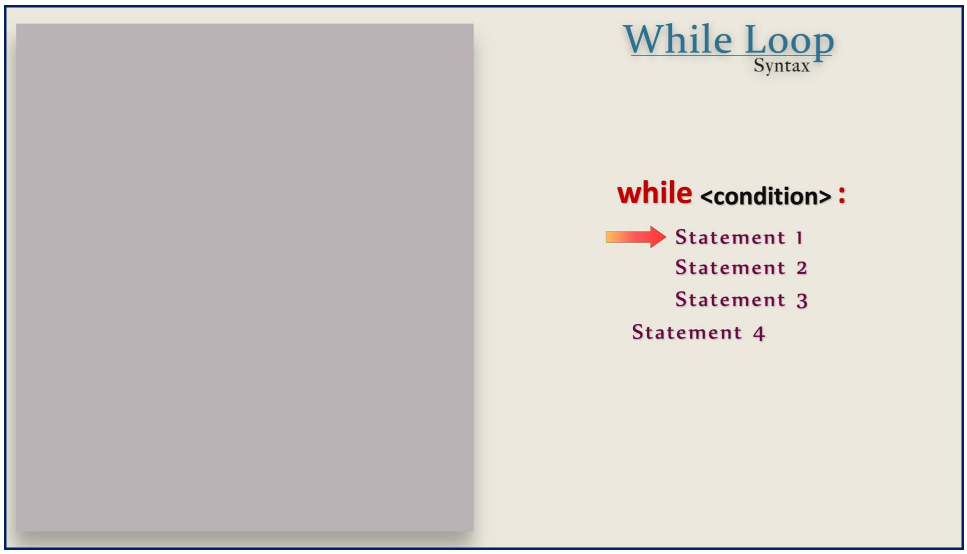
Statement 1
Statement 2
Statement 3
Statement 4

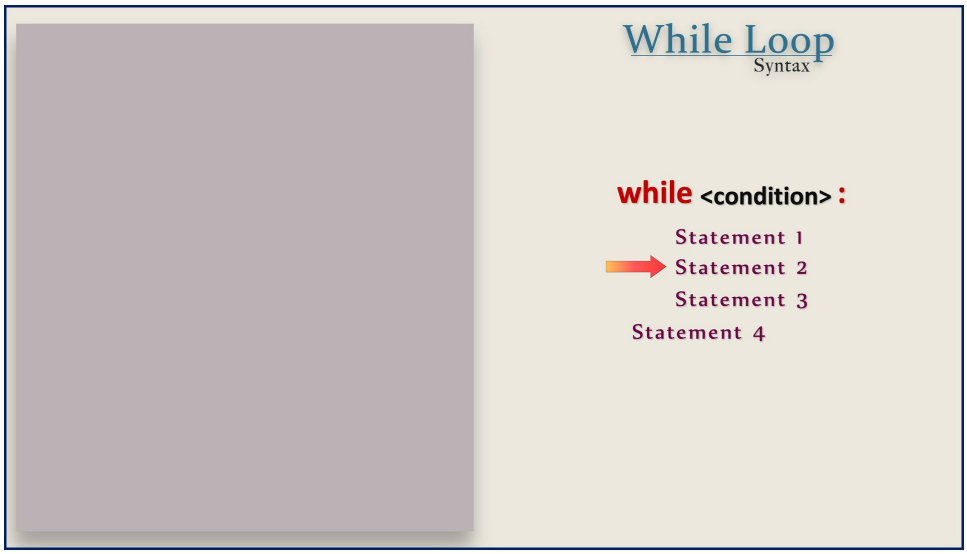
While Loop
Syntax

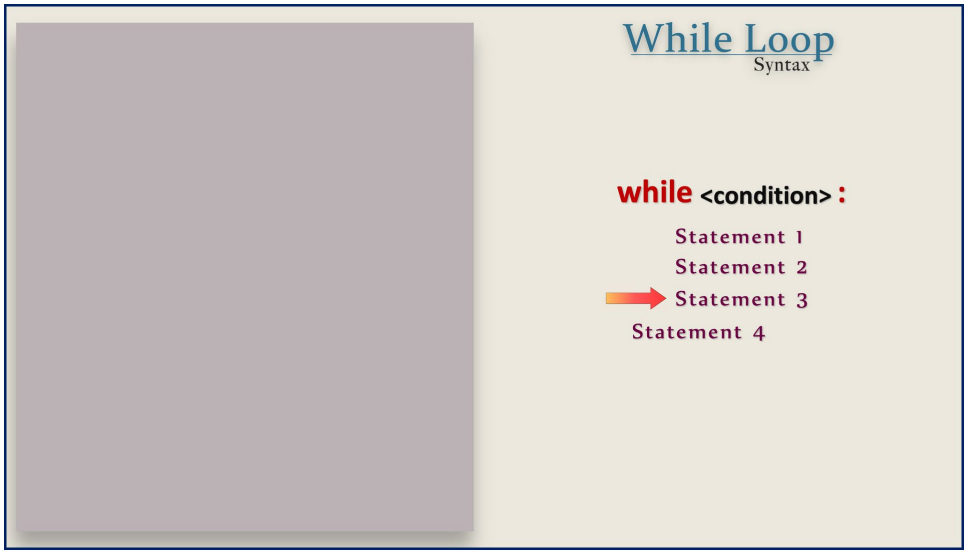
while <condition> **:**

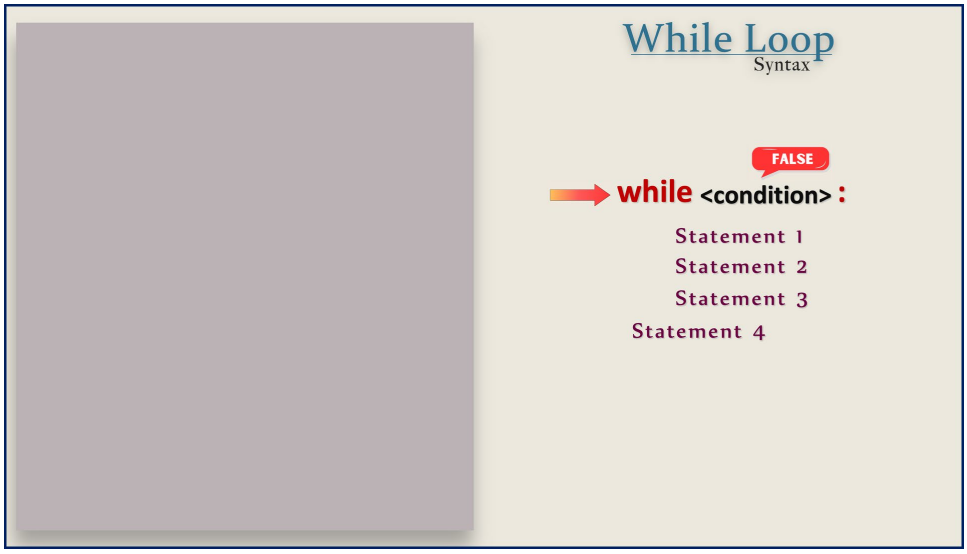
Statement 1
Statement 2
Statement 3
Statement 4


While Loop
Syntax
TRUE
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4

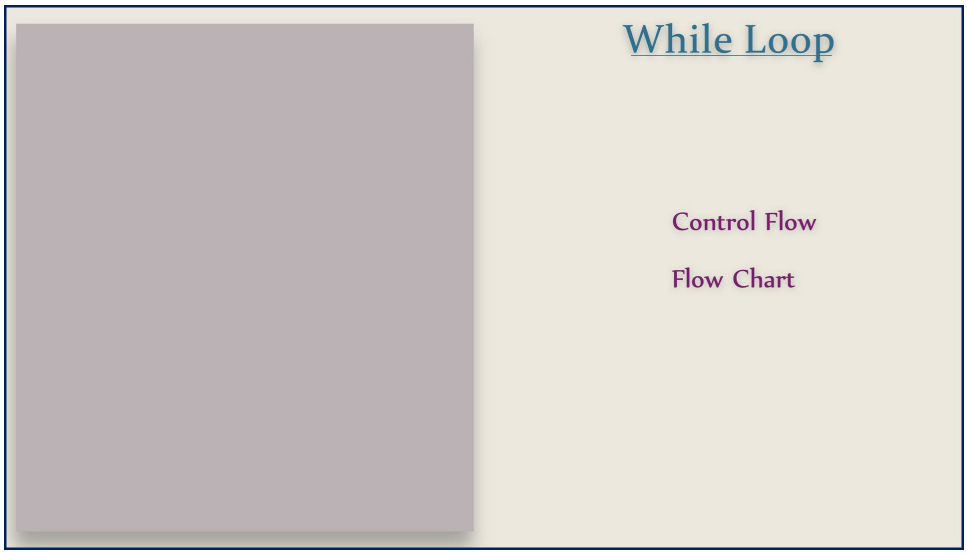

While Loop
Syntax
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4

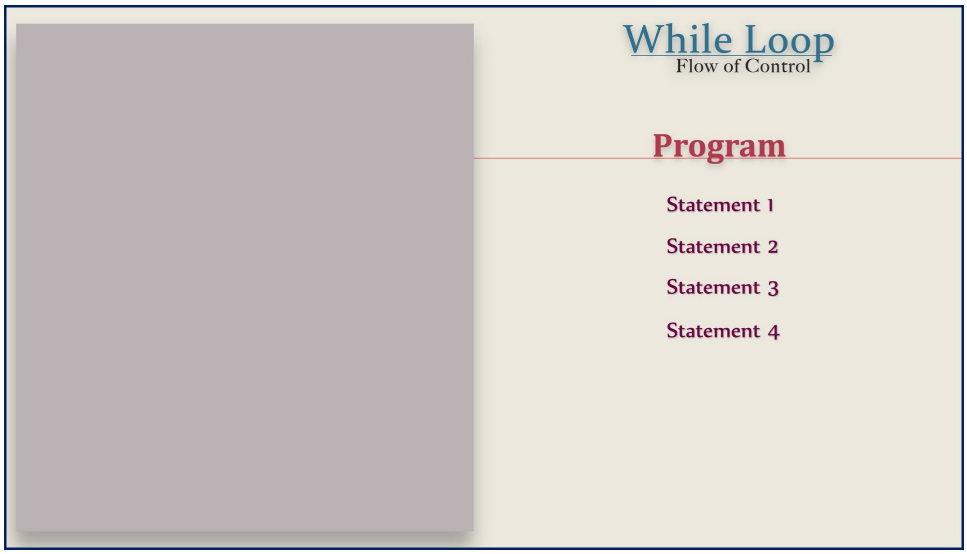

While Loop
Syntax
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4

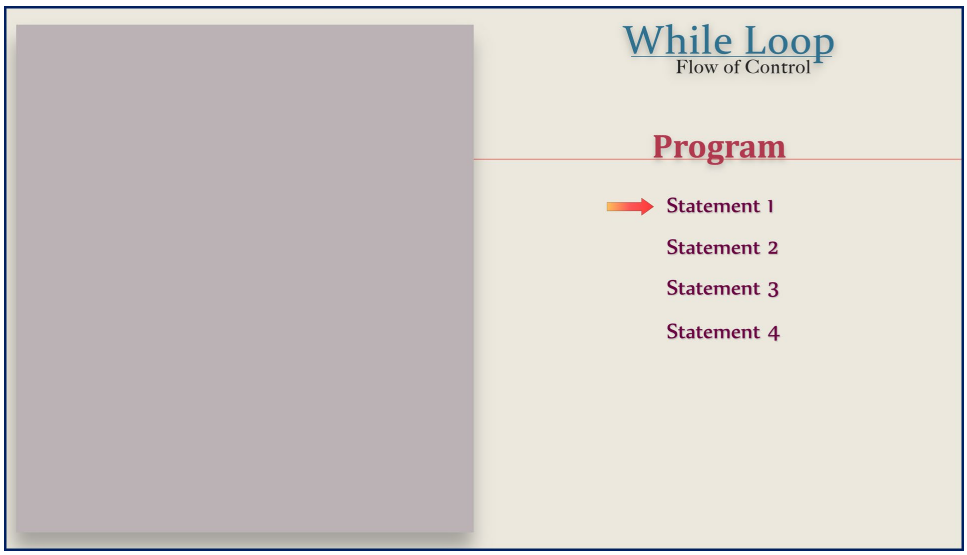

While Loop
Syntax
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4

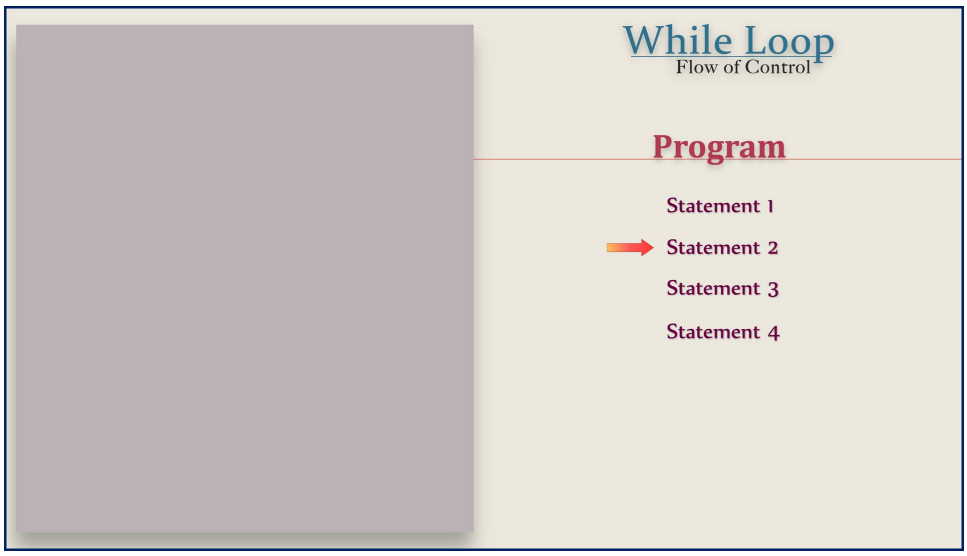

While Loop
Syntax
FALSE
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4

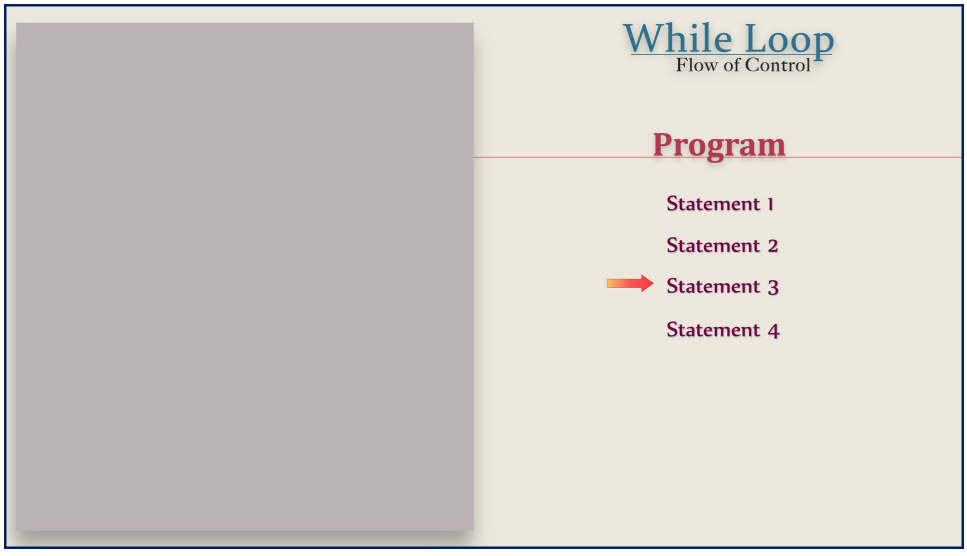

While Loop
Syntax
FALSE
while <condition> :
Statement 1
Statement 2
Statement 3
Statement 4


While Loop
Control Flow
Flow Chart


While Loop
Flow of Control
Program
Statement 1
Statement 2
Statement 3
Statement 4


While Loop
Flow of Control
Program
Statement 1
Statement 2
Statement 3
Statement 4


While Loop
Flow of Control
Program
Statement 1
Statement 2
Statement 3
Statement 4


While Loop
Flow of Control
Program
Statement 1
Statement 2
Statement 3
Statement 4

While Loop

Flow of Control

Program

Statement 1

Statement 2

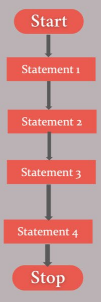
Statement 3

Statement 4

While Loop

Flow of Control

Flow Chart



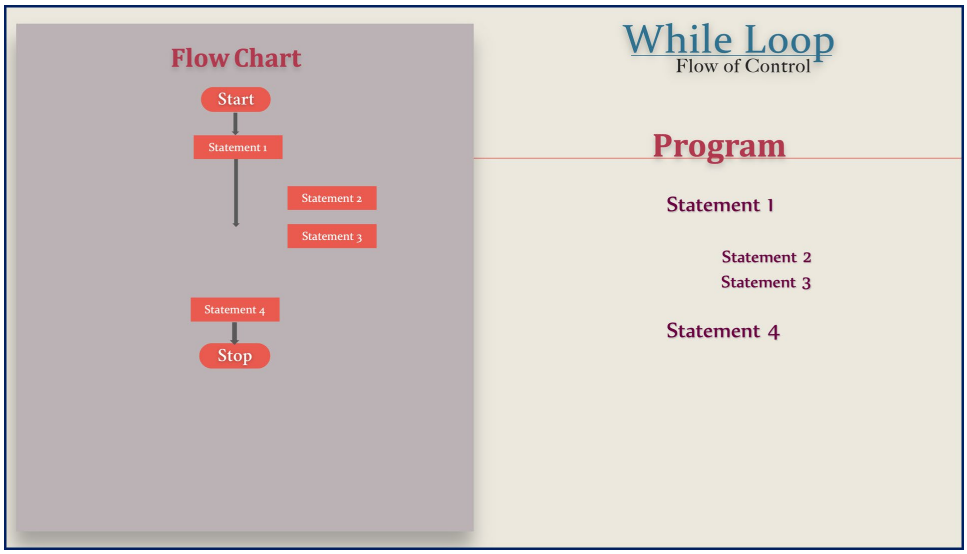
Program

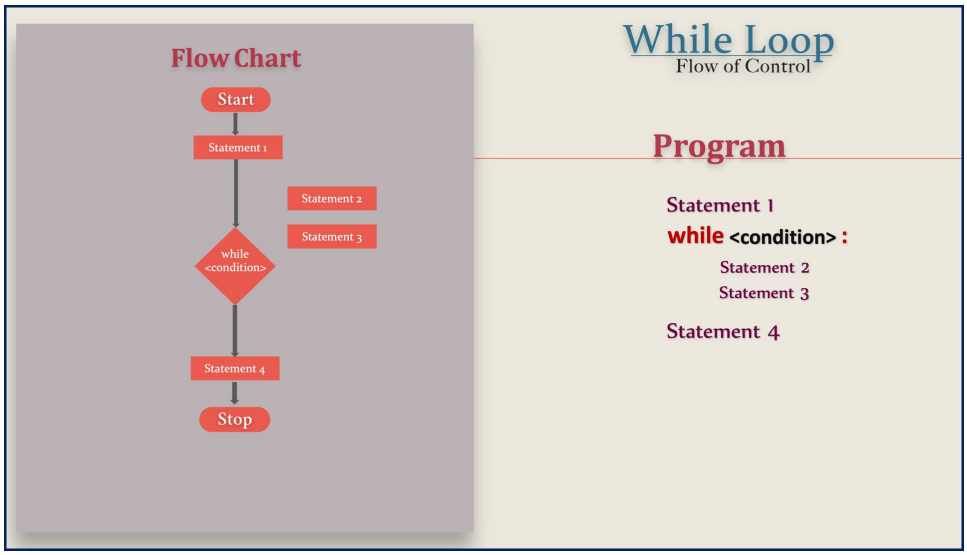
Statement 1

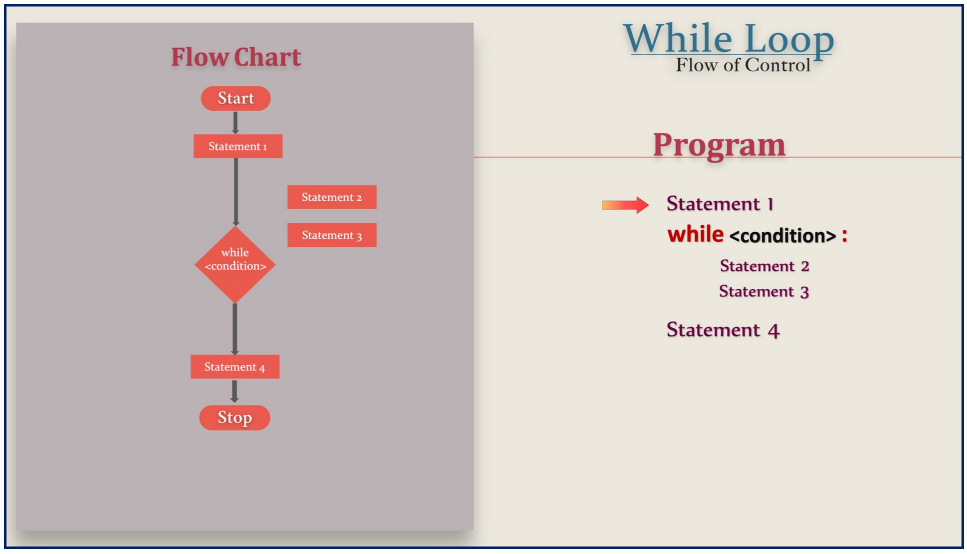
Statement 2

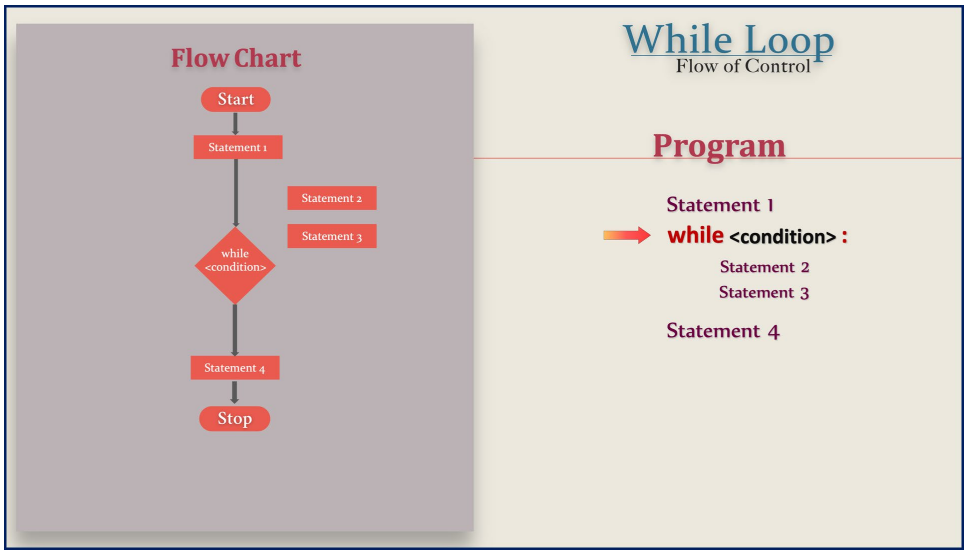
Statement 3

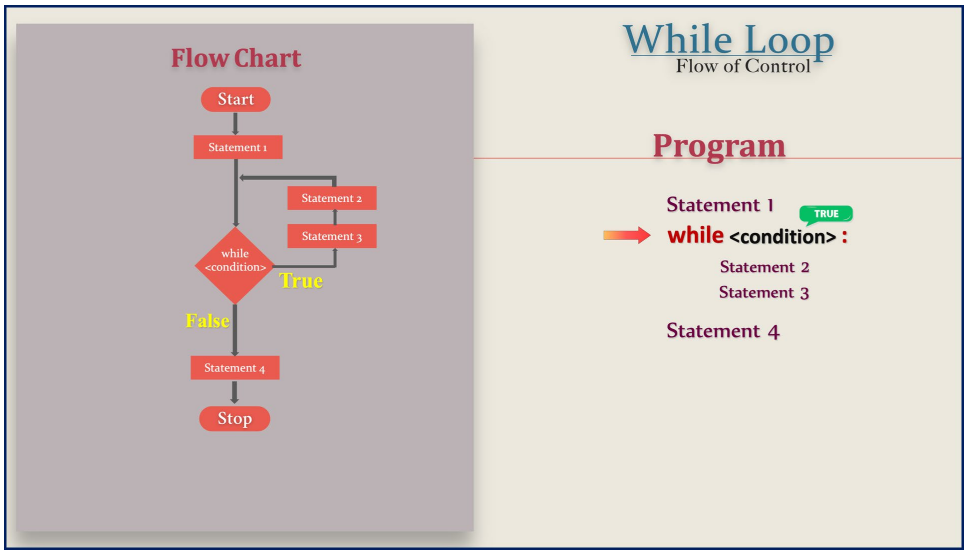
Statement 4


Flow Chart
Start
Statement 1
Statement 2
Statement 3
Statement 4
Stop
While Loop
Flow of Control
Program
Statement 1
Statement 2
Statement 3
Statement 4


Flow Chart
Start
Statement 1
Statement 2
Statement 3
while <condition>
Statement 4
Stop
While Loop
Flow of Control
Program
Statement 1
while <condition> :
Statement 2
Statement 3
Statement 4

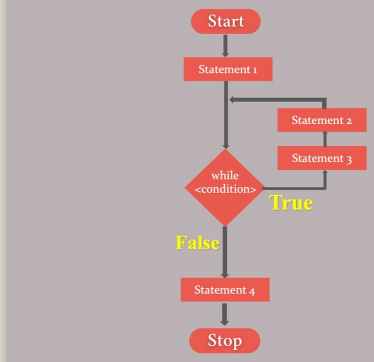

Flow Chart
Start
Statement 1
Statement 2
Statement 3
while <condition>
Statement 4
Stop
While Loop
Flow of Control
Program
Statement 1
while <condition> :
Statement 2
Statement 3
Statement 4


Flow Chart
Start
Statement 1
Statement 2
Statement 3
while <condition>
Statement 4
Stop
While Loop
Flow of Control
Program
Statement 1
while <condition> :
Statement 2
Statement 3
Statement 4


Flow Chart
Start
Statement 1
Statement 2
Statement 3
while <condition>
True
False
Statement 4
Stop
While Loop
Flow of Control
Program
Statement 1
TRUE
while <condition> :
Statement 2
Statement 3
Statement 4

While Loop
Flow of Control

Flow Chart

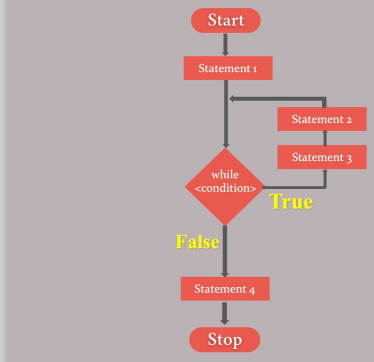


Program

```
Statement 1
while <condition> :
    Statement 2
    Statement 3
Statement 4
```

While Loop
Flow of Control

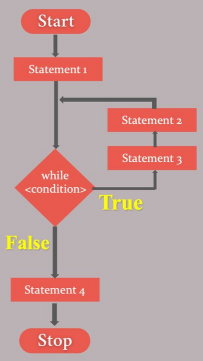
Flow Chart



Program

```
Statement 1
while <condition> :
    Statement 2
    Statement 3
Statement 4
```

Flow Chart



While Loop

Flow of Control

Program

Statement 1

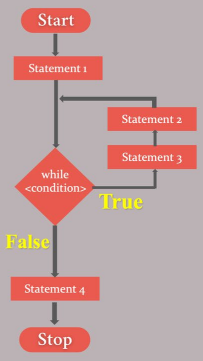
while <condition> :

Statement 2

Statement 3

Statement 4

Flow Chart



While Loop

Flow of Control

Program

Statement 1 FALSE

while <condition> :

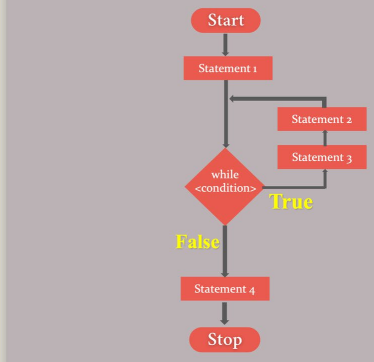
Statement 2

Statement 3

Statement 4

While Loop
Flow of Control

Flow Chart

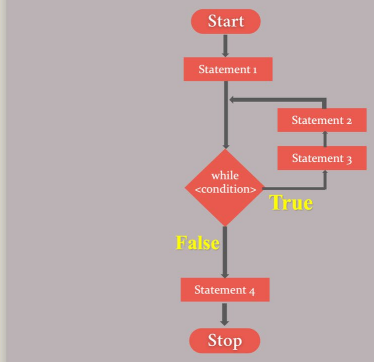


Program

Statement 1
while <condition> :
Statement 2
Statement 3
→ Statement 4

While Loop
Flow of Control

Flow Chart



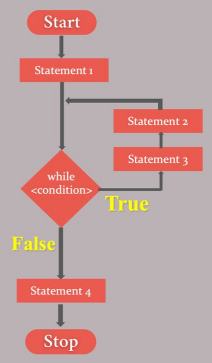
Program

Statement 1
while <condition> :
Statement 2
Statement 3
Statement 4
→

While Loop

Flow of Control

Flow Chart



Program

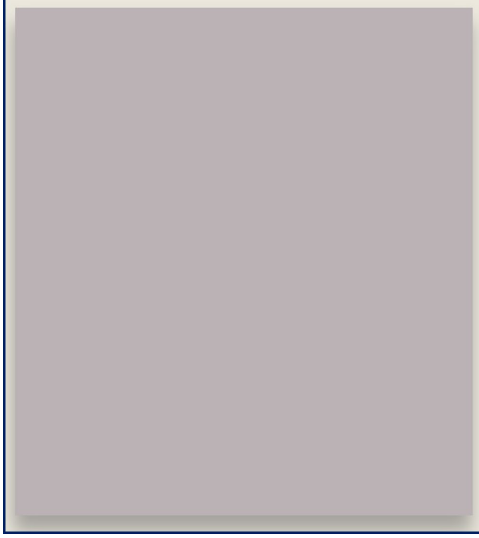
```
Statement 1
while <condition> :
    Statement 2
    Statement 3
Statement 4
```

While Loop

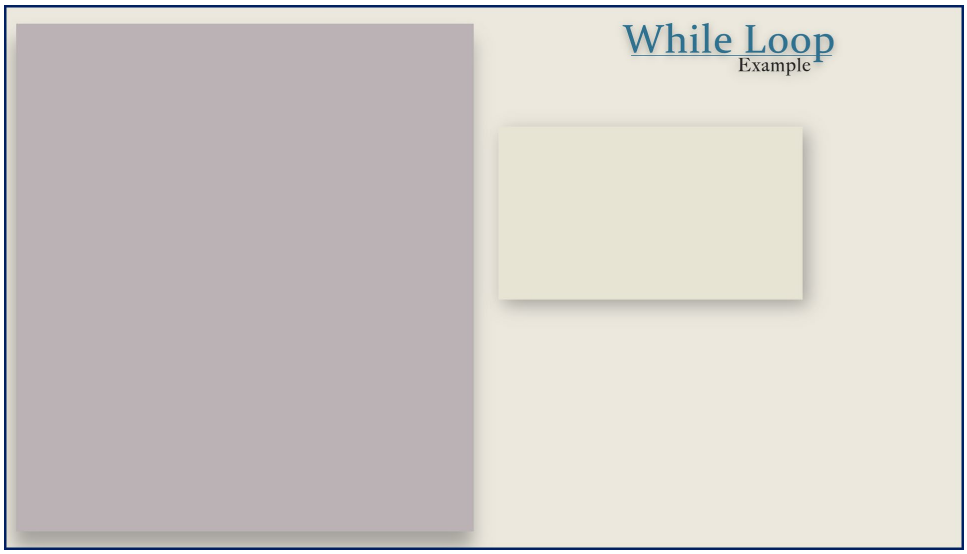
Example

While Loop

Example



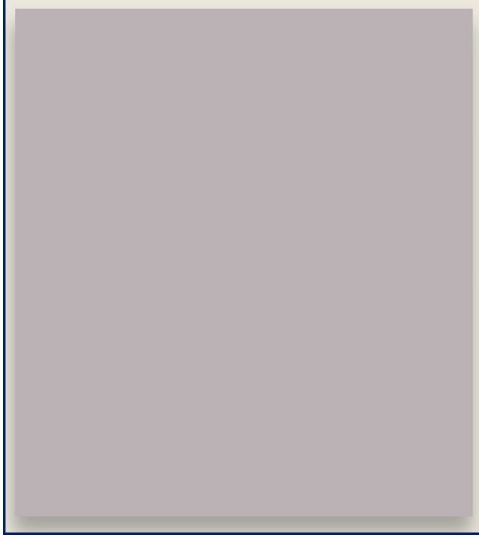
While Loop
Example



While Loop
Example

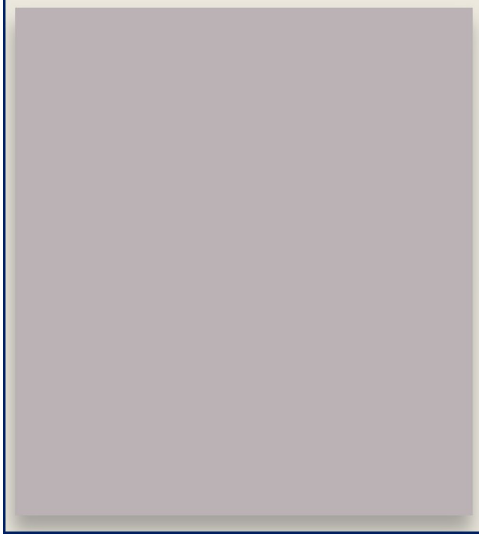
n = 5

While Loop
Example



```
n = 5
print( n )
```

While Loop
Example



```
n = 5
print( n )
n = n — 1
```

While Loop
Example

```
n = 5
print( n )
n = n — 1
```

5

While Loop
Example

```
n = 5
print( n )
n = n — 1
```

5 – 1

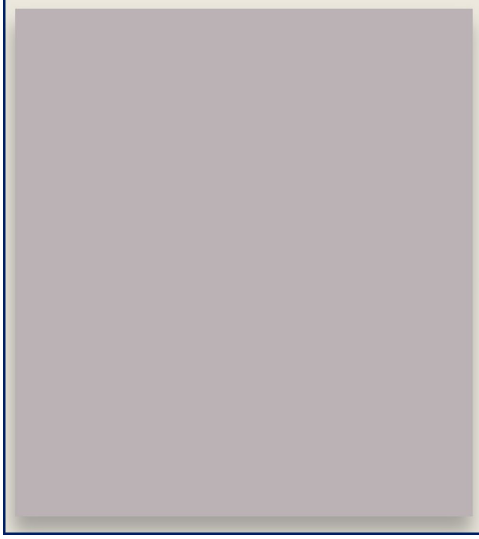
While Loop
Example

```
n = 5
print( n )
n = n — 1
4 = 5 – 1
```

While Loop
Example

```
n = 5
print( n )
n = n — 1   Counter
4 = 5 – 1
```

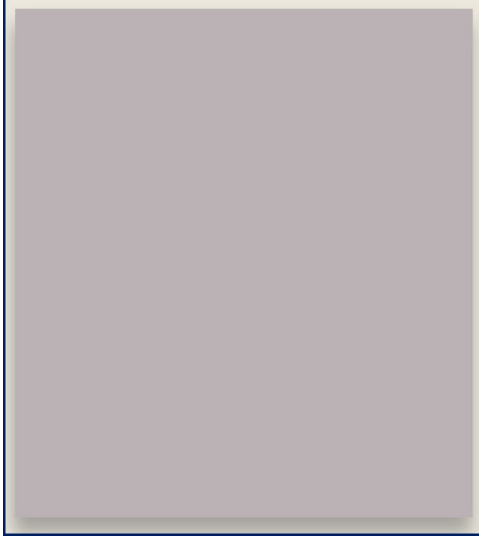
While Loop
Example



```
n = 5

print( n )
n = n — 1
```

While Loop
Example



```
n = 5
while       :
    print( n )
    n = n — 1
```

While Loop

Example

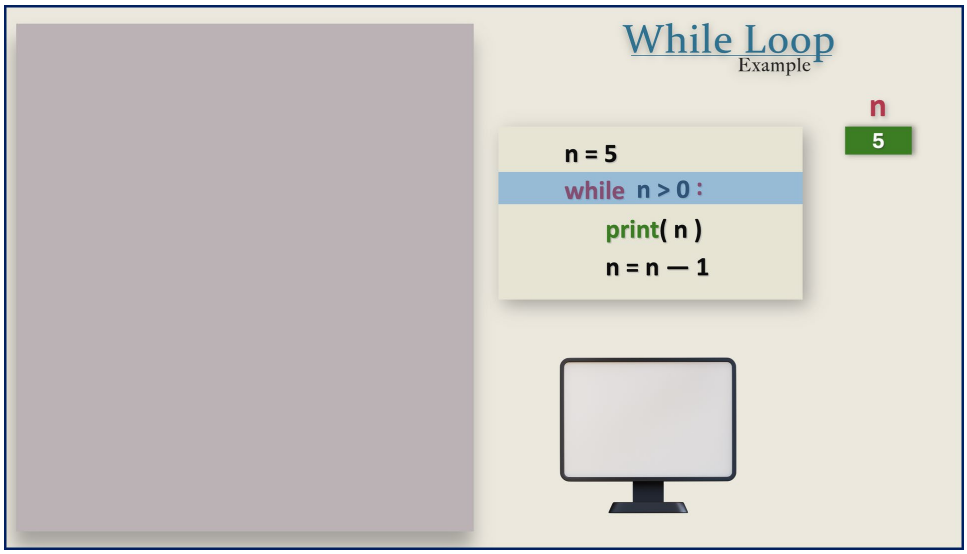
```
n = 5
while n > 0 :
    print( n )
    n = n — 1
```

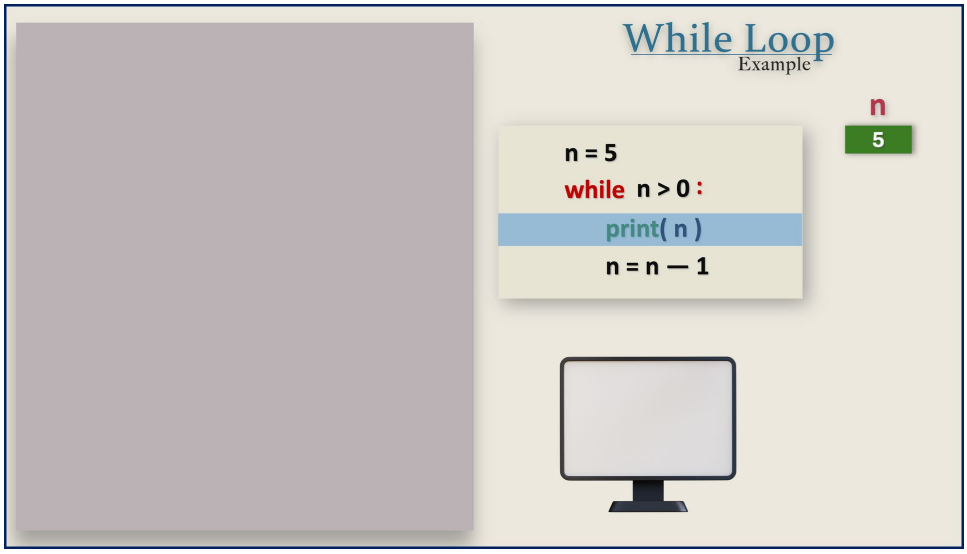
While Loop
Example

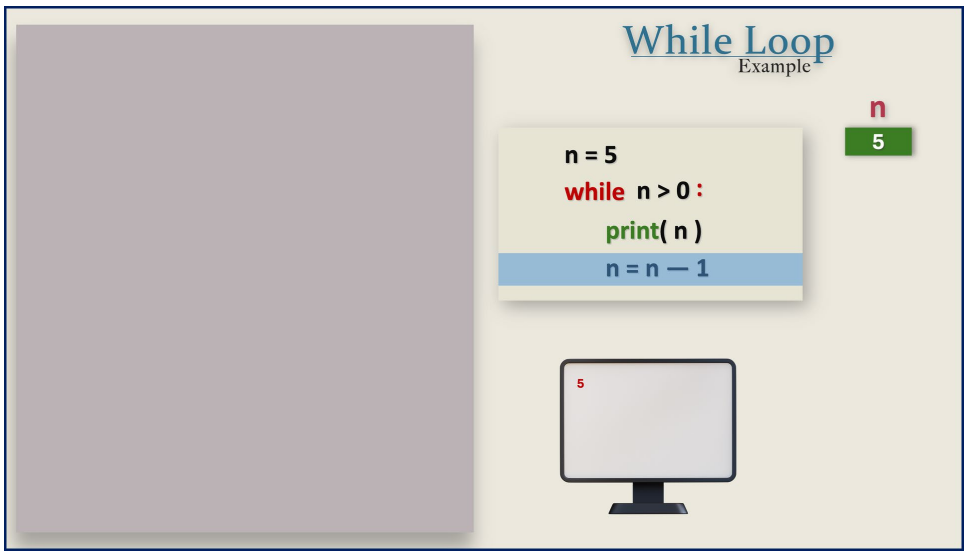
```
n = 5
while n > 0 :
    print( n )
    n = n — 1
```

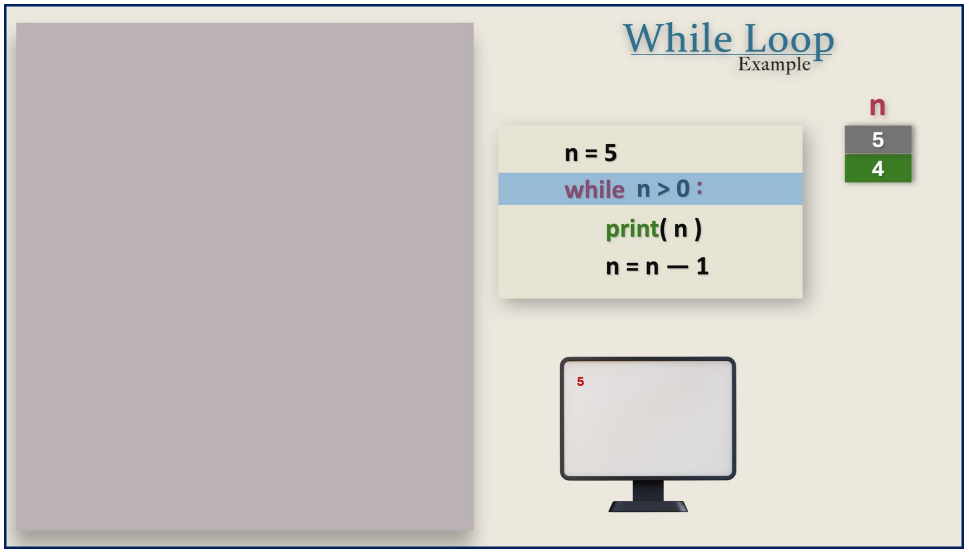
While Loop
Example

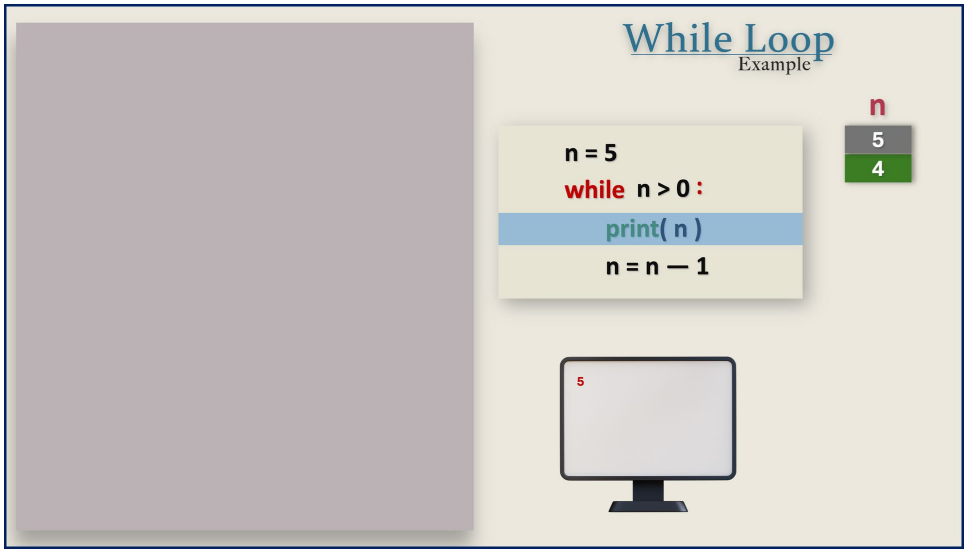
```
n = 5
while n > 0 :
    print( n )
    n = n — 1
```

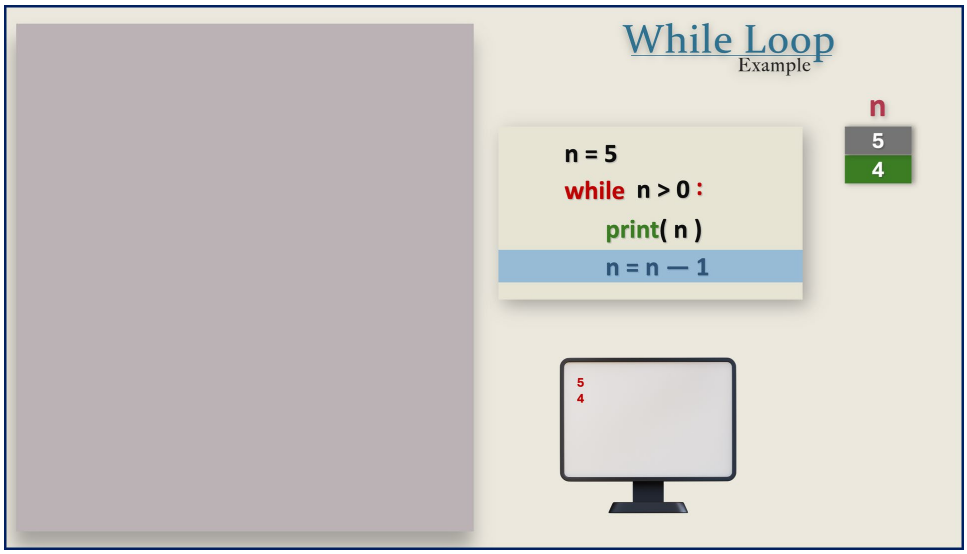

While Loop
Example
n
5
n = 5
while n > 0 :
print(n)
n = n — 1

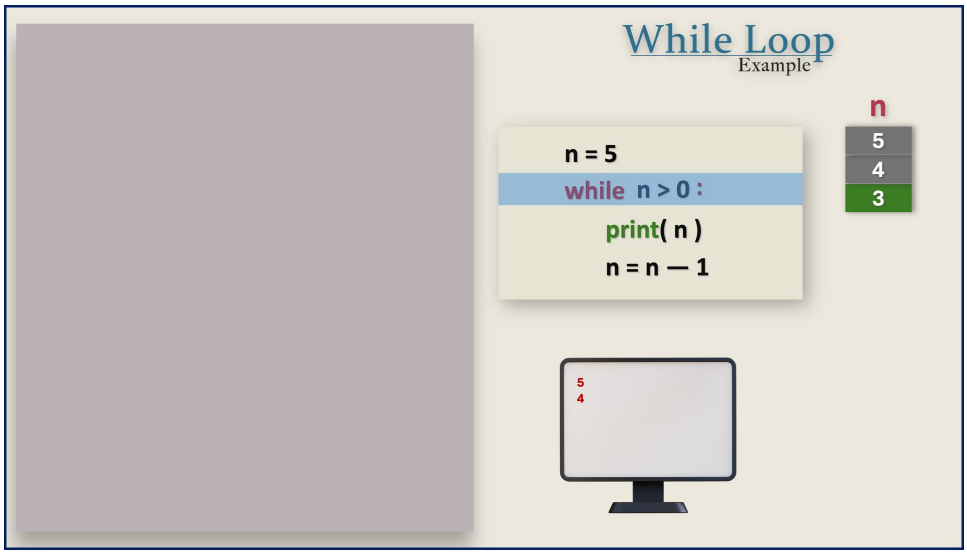

While Loop
Example
n
5
n = 5
while n > 0 :
print(n)
n = n — 1

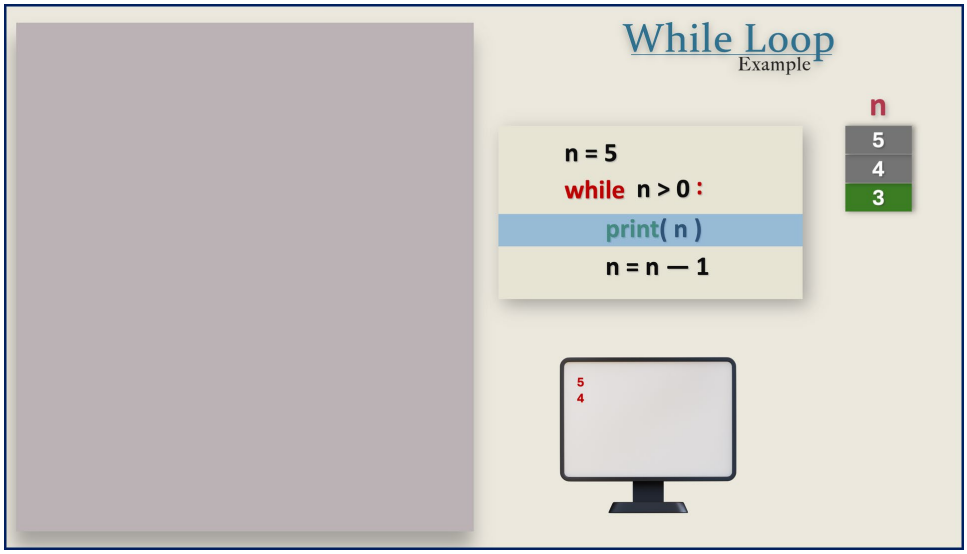

While Loop
Example
n
5
n = 5
while n > 0 :
print(n)
n = n — 1
5

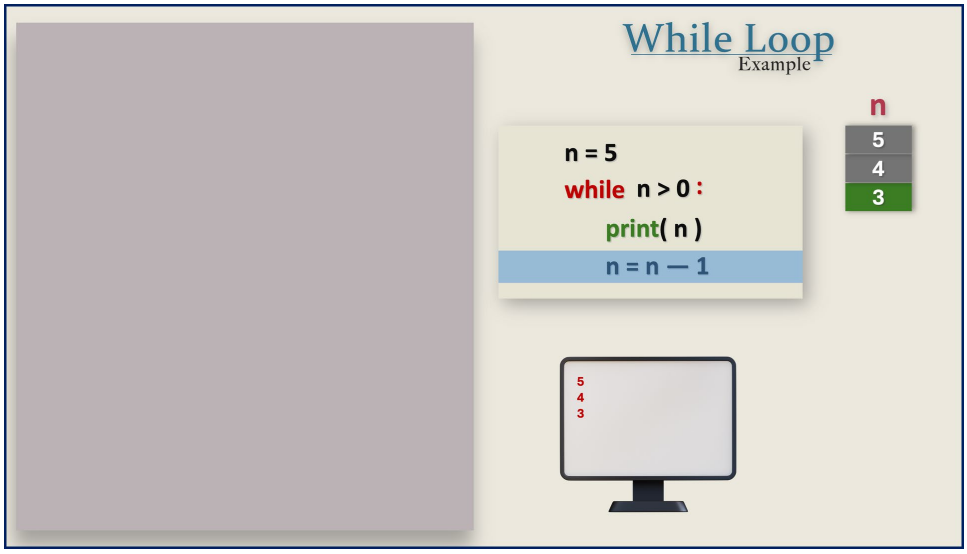

While Loop
Example
n
5
4
n = 5
while n > 0 :
print(n)
n = n — 1
5

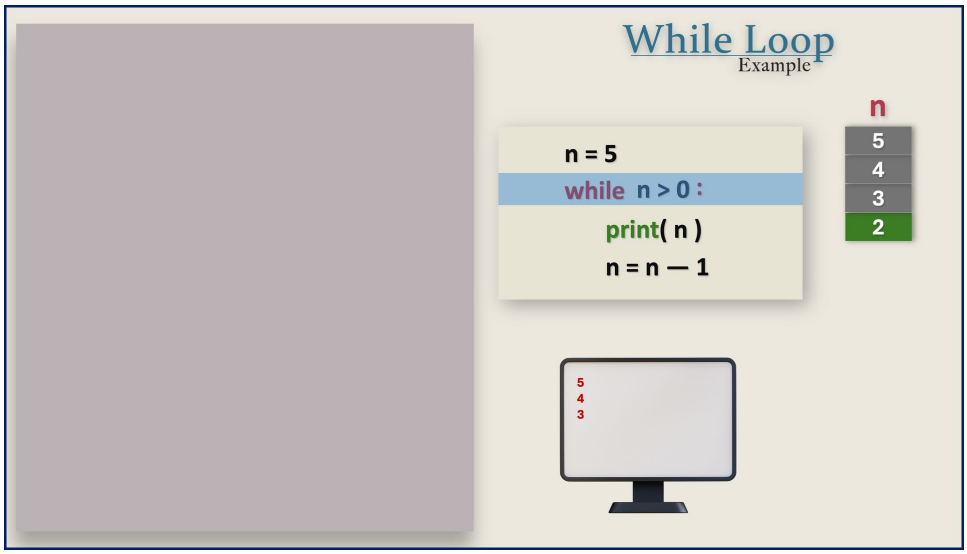

While Loop
Example
n
5
4
n = 5
while n > 0 :
print(n)
n = n — 1
5

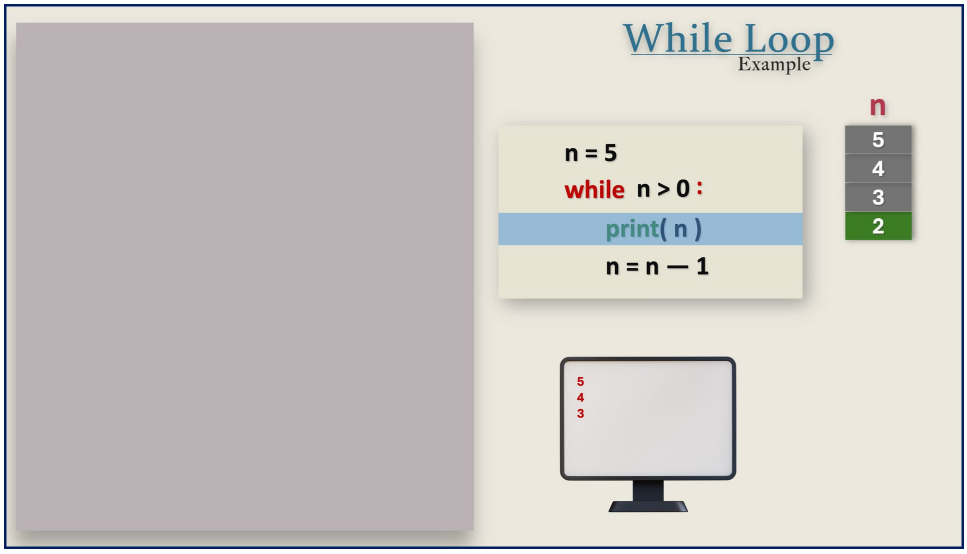

While Loop
Example
n
5
4
n = 5
while n > 0 :
print(n)
n = n — 1
5
4

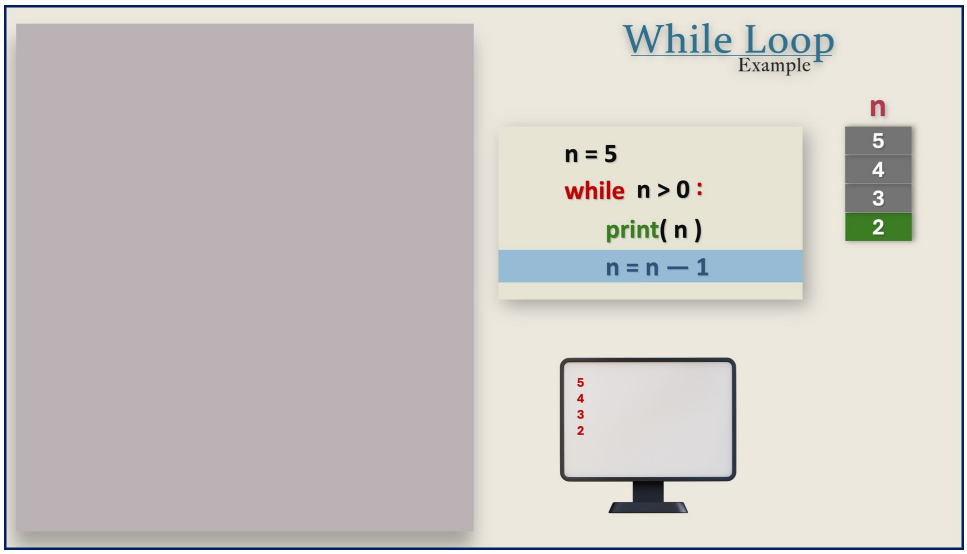

While Loop
Example
n
5
4
3
n = 5
while n > 0 :
print(n)
n = n — 1
5
4

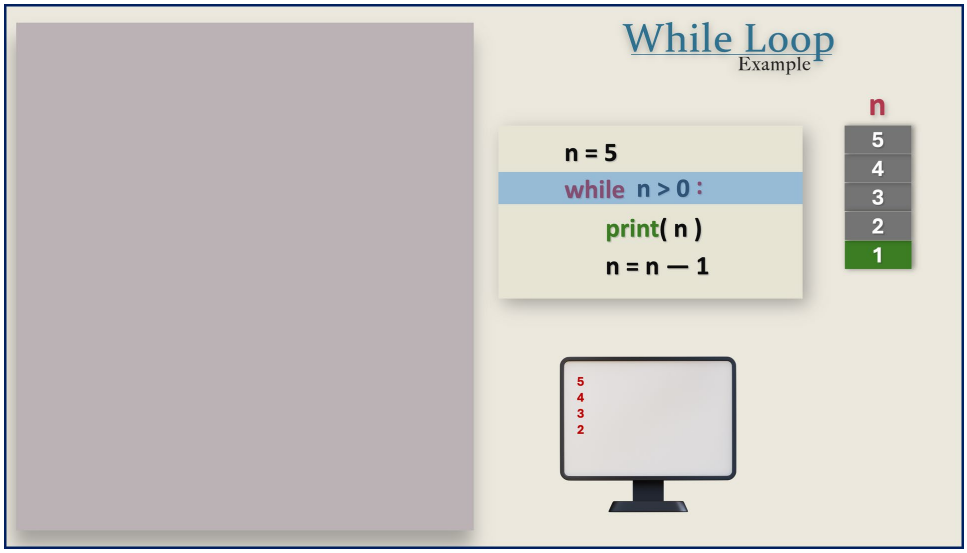

While Loop
Example
n
5
4
3
n = 5
while n > 0 :
print(n)
n = n — 1
5
4

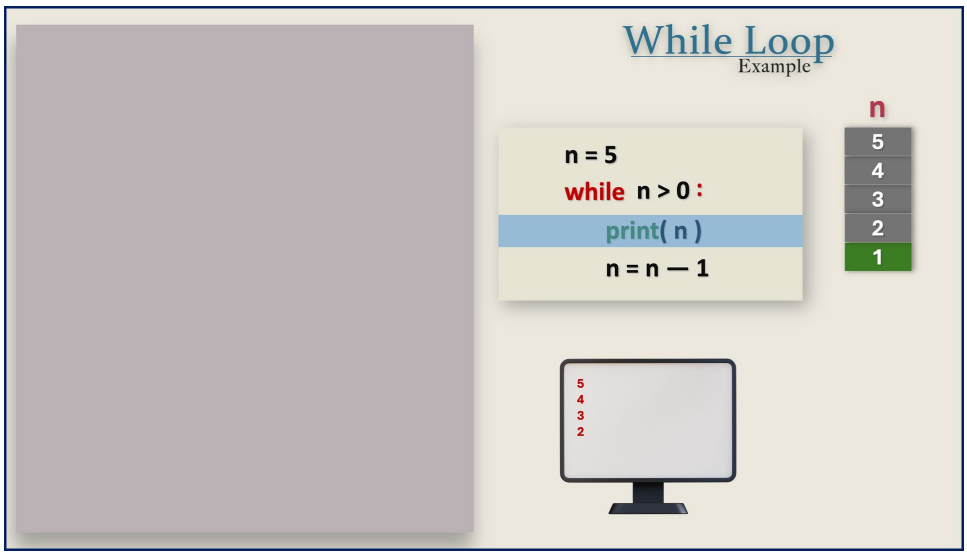

While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
5
4
3

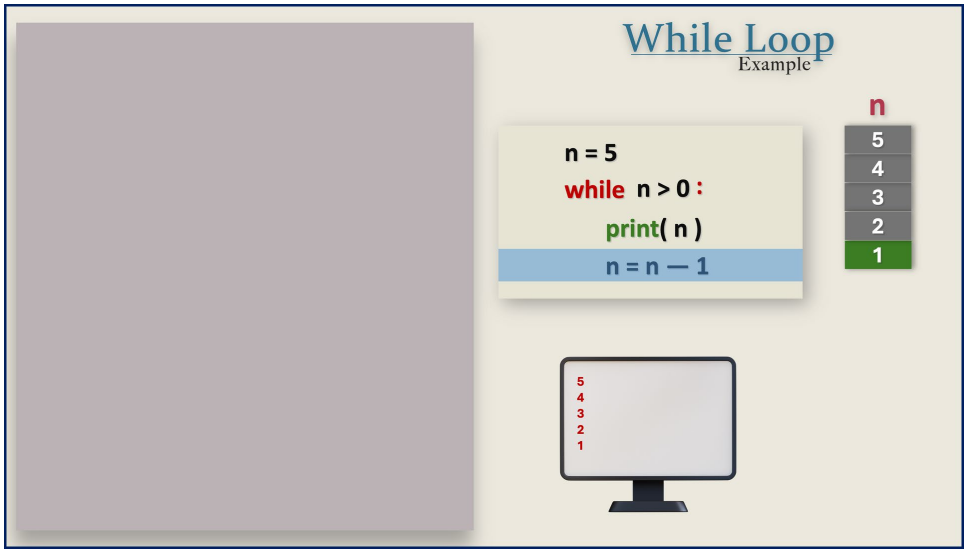

While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
5
4
3

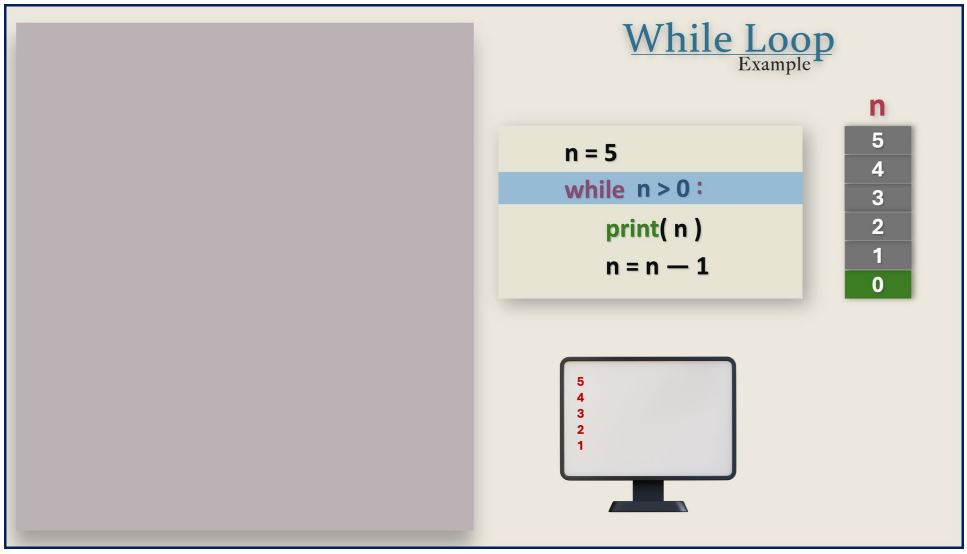

While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
5
4
3

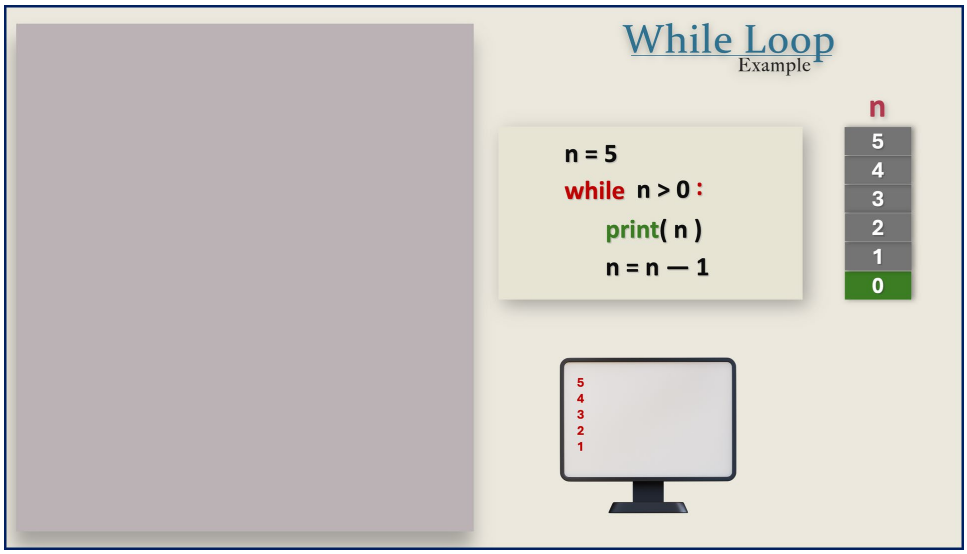

While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
5
4
3
2


While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
1
5
4
3
2


While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
1
5
4
3
2


While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
1
5
4
3
2
1


While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
1
0
5
4
3
2
1


While Loop
Example
n = 5
while n > 0 :
print(n)
n = n — 1
n
5
4
3
2
1
0
5
4
3
2
1